LAB 7

Fuzzing for Input Validation Bugs

Cloud Storage Tier

PES1UG24CS317

Course / Lab	Lab 7 – Fuzzing for Input Validation Bugs
Project repository	L6_L7_12_cloud_storage_tier
Testing framework	Hypothesis and pytest
Python environment	Python 3.13.15

1. Objective

The objective of this lab is to use fuzz testing as a security and quality-assurance technique. The Hypothesis property-based testing library generates a range of unexpected and malformed inputs to identify input-validation bugs in the supplied Python application. The application is then hardened and the tests are executed again to verify the fixes.

2. Tools and Environment

- Programming language: Python 3.13.15
- Test runner: pytest 9.2.0
- Fuzzing library: Hypothesis 6.170.15
- Installation command: py -m pip install hypothesis pytest
- Test command: py -m pytest -v test_processor.py
- Application command: py processor.py

3. Application Functions Tested

The Lab 7 processor.py file contains three functions:

- **parse_storage_quota() –** Parses a storage quota with MB, GB or TB units and converts it to GB.
- **compute_compression_ratio() –** Calculates the ratio of raw bytes to compressed bytes.
- **lookup_bandwidth_tier_rate() –** Looks up the bandwidth/egress rate for supported storage-tier descriptors.

4. Procedure and Test Evidence

4.1 Running the Original Application

The original application was executed before making changes. Valid sample values were entered to confirm the normal execution path: 500MB for quota, 1024:256 for the compression metric, and HOT_TIER for the tier descriptor.

Observed output:

- 500MB → Quota: 0.50 GB
- 1024:256 → Compression Ratio: 4.0
- HOT_TIER → Rate per GB: $0.08

```
C:\Users\admin\Downloads\L6_L7_12_cloud_storage_tier-main\L6_L7_12_cloud_st
orage_tier-main\Lab 7>py -m pytest --version
pytest 9.2.0

C:\Users\admin\Downloads\L6_L7_12_cloud_storage_tier-main\L6_L7_12_cloud_st
orage_tier-main\Lab 7>py processor.py
==== Running parse_storage_quota ====
Enter quota (e.g. 500MB, 10GB, 2TB): 500MB
Quota: 0.50 GB

==== Running compute_compression_ratio ====
Enter raw:compressed bytes (e.g. 1024:256): 1024:256
Compression Ratio: 4.0

==== Running lookup_bandwidth_tier_rate ====
Enter tier descriptor (e.g. HOT_TIER, COLD, ARCHIVE): HOT_TIER
Rate per GB: $ 0.08

C:\Users\admin\Downloads\L6_L7_12_cloud_storage_tier-main\L6_L7_12_cloud_st
orage_tier-main\Lab 7>
```

Figure 1. Original application output
The original processor.py was executed with valid sample inputs: 500MB, 1024:256 and HOT_TIER.

4.2 Writing Hypothesis Tests

The test_processor.py file was completed with three property-based tests. Each test uses @given(st.text() | st.none()) to generate arbitrary text values and None. The tests call the relevant function and fail if an unexpected exception is raised.

```
from hypothesis import given, strategies as st
from processor import (parse_storage_quota, compute_compression_ratio,
                       lookup_bandwidth_tier_rate)

@given(st.text() | st.none())
def test_parse_storage_quota_no_crash(s):
    try:
        parse_storage_quota(s)
    except Exception as e:
        assert False, f"parse_storage_quota crashed with: {e}"

@given(st.text() | st.none())
def test_compute_compression_ratio_safe(s):
    try:
        compute_compression_ratio(s)
    except Exception as e:
        assert False, f"compute_compression_ratio crashed with: {e}"

@given(st.text() | st.none())
def test_lookup_bandwidth_tier_rate_safe(s):
    try:
        lookup_bandwidth_tier_rate(s)
    except Exception as e:
        assert False, f"lookup_bandwidth_tier_rate crashed with: {e}"
```

4.3 Fuzzing the Original Application

The completed tests were run against the original processor.py. All three tests failed, demonstrating that Hypothesis had discovered inputs that caused the original functions to raise exceptions.

Examples of minimized failing inputs and observed issues:

Function	Generated input	Observed failure
parse_storage_quota()	Empty string ('')	ValueError during integer conversion
parse_storage_quota()	None	AttributeError when calling strip()
compute_compression_ratio()	Empty/malformed text such as ':'	ValueError or unpacking error
compute_compression_ratio()	None	AttributeError when calling split()
lookup_bandwidth_tier_rate()	Empty string ('')	KeyError for an empty prefix
lookup_bandwidth_tier_rate()	None	AttributeError when calling strip()

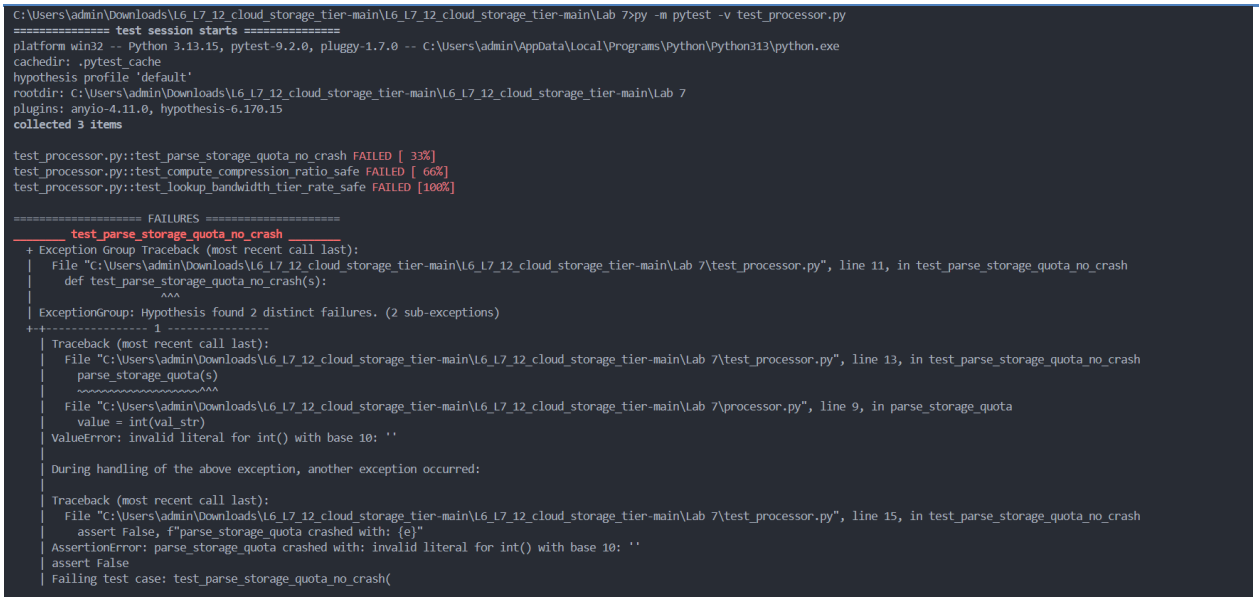


Figure 2. Hypothesis failures – quota validation

Hypothesis exposed invalid handling of an empty string and None in parse_storage_quota(), along with malformed numeric input.

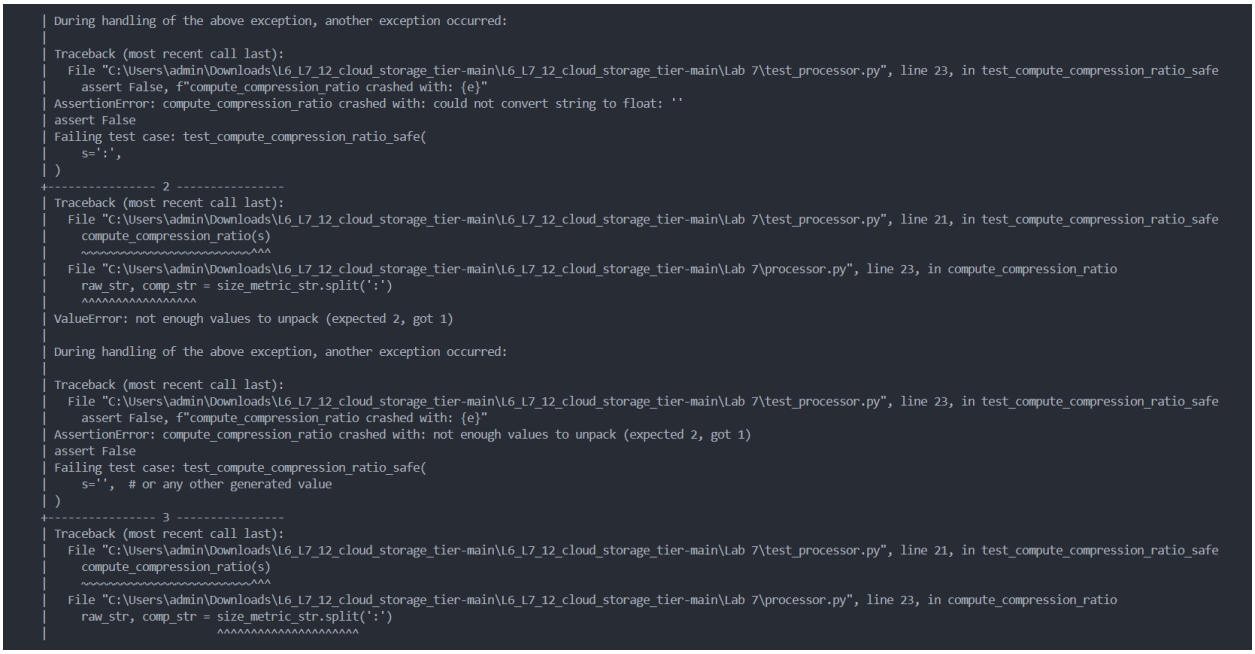


Figure 3. Hypothesis failures – compression and tier lookup
Hypothesis exposed malformed compression input and invalid tier descriptors, including an empty string.

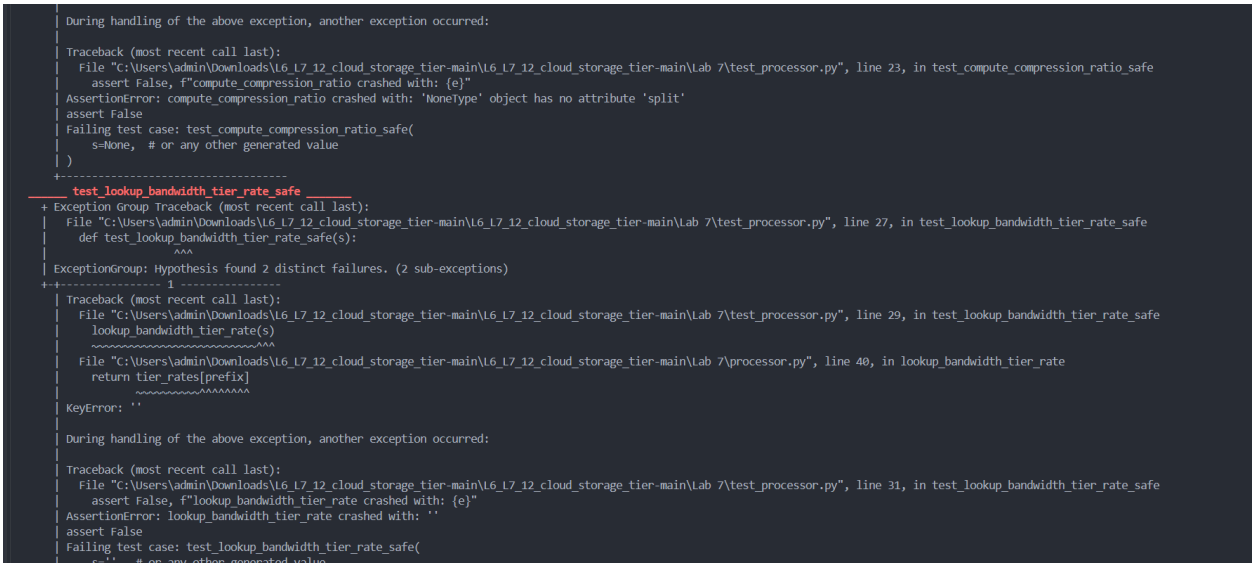


Figure 4. Hypothesis failures – None input
The failure output shows AttributeError when None is passed to functions that call string methods.

4.4 Fixing the Application

The three functions were hardened with defensive input validation. The corrected implementation checks input types before using string methods, validates the expected input format, handles invalid numeric conversions, rejects non-finite values, prevents division by zero, and uses a safe dictionary lookup for unsupported tier prefixes. Invalid inputs return None rather than causing an uncaught exception.

- **parse_storage_quota() –** Check that input is a string; trim and uppercase it; validate the complete number-plus-unit format before conversion; return None for invalid input.
- **compute_compression_ratio() –** Require a string with exactly two colon-separated fields; parse finite numeric values; reject negative raw bytes and non-positive compressed bytes; handle conversion and division errors.
- **lookup_bandwidth_tier_rate() –** Check input type and minimum descriptor length; normalize case and whitespace; use dict.get() so unsupported prefixes return None instead of raising KeyError.

4.5 Rerunning the Tests and Application

After updating processor.py, the same Hypothesis test suite was run again. All three tests passed. The fixed interactive application was also run with the same valid sample values, and it continued to produce the expected results.

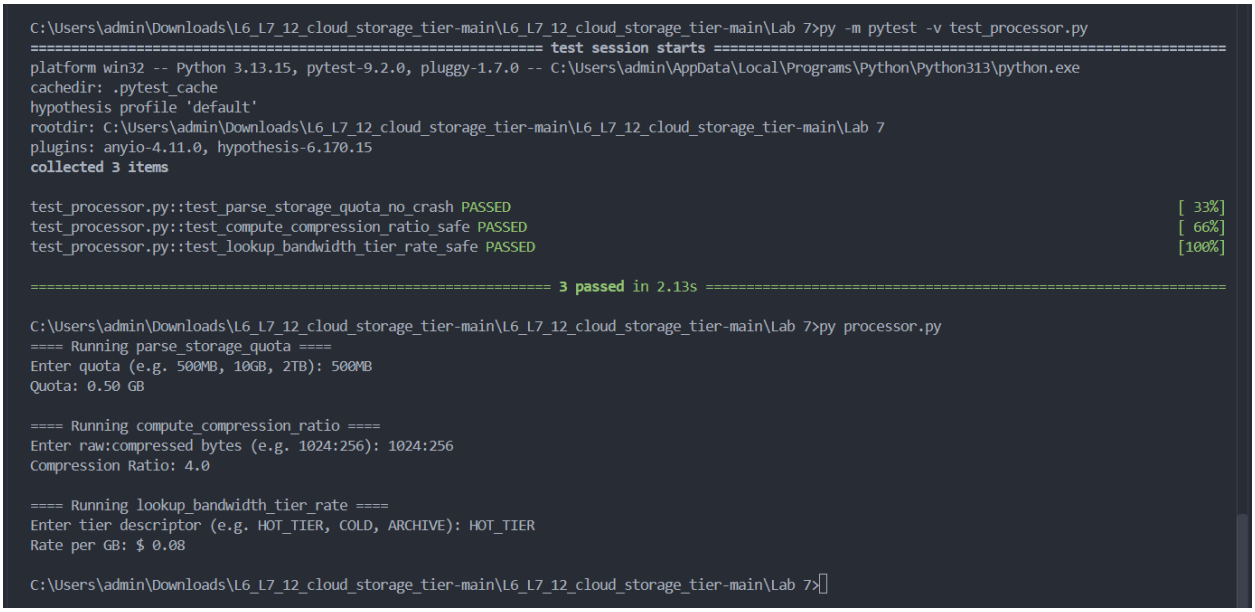


Figure 5. Final passing tests and application output
After the fixes, all three Hypothesis tests passed. The application also returned the expected results for the valid sample inputs.

5. Result

The fuzz tests identified input-validation weaknesses in all three functions in the original application. After defensive checks and error handling were added, the test run completed successfully with 3 passed in 2.13 seconds. The valid sample inputs continued to return 0.50 GB, a compression ratio of 4.0, and a HOT tier rate of $0.08.

6. Conclusion

This lab demonstrated how property-based fuzz testing can reveal edge cases that may not be covered by ordinary example-based testing. Hypothesis generated unexpected values such as empty strings and None, which exposed crashes in the original implementation. Defensive input validation was added to the application, and the successful rerun of all three tests verified that the identified crash cases were handled.

7. Deliverables Checklist

- Fixed processor.py
- Updated test_processor.py containing Hypothesis tests
- This Word report with screenshots of the original run, fuzz-test failures, and final passing run